

fortranspire: an LLM-driven Model Context Protocol pipeline for porting legacy Fortran HPC kernels to GPU (OpenACC) and differentiable JAX

Loïc Maurin ¹✉

¹ Independent Researcher, Toulouse, France ✉ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 24 June 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

fortranspire is an open-source pipeline that transforms legacy Fortran 90 scientific code into GPU-accelerated, Python-callable, and optionally differentiable form. It combines deterministic abstract-syntax-tree (AST) analysis with targeted Large Language Model (LLM) calls, exposed through a Model Context Protocol (MCP) ([Anthropic, 2024](#)) server so the pipeline can be driven from an editor, a notebook, or a continuous-integration job. A run produces PURE/ELEMENTAL kernels with explicit INTENT, the same kernels annotated with OpenACC ([OpenACC-Standard.org, 2022](#)) pragmas, a Cython ([Behnel et al., 2011](#)) wrapper compiled through scikit-build, and, on demand, a JAX ([Bradbury et al., 2018](#)) translation that is differentiable and JIT-compileable by XLA. The AST stage uses Loki ([ECMWF, 2024](#)); the LLM stage uses any OpenAI-compatible endpoint — Mistral La Plateforme ([Mistral AI, 2024b](#)) with Codestral ([Mistral AI, 2024a](#)) for code generation and Mistral Large 2 ([Mistral AI, 2024c](#)) for semantic reasoning by default, or a self-hosted vLLM ([Kwon et al., 2023](#)) / TGI / Ollama server.

Statement of need

Scientific HPC codebases are dominated by Fortran 90 written between the late 1980s and the early 2000s. Production codes for seismic imaging, numerical weather prediction (ARPEGE, AROME, IFS) ([ECMWF, 2023](#)), computational fluid dynamics, and reservoir simulation run for days on CPU clusters while modern GPU accelerators sit underused because porting these codes by hand is slow (2–6 weeks per kernel for a senior HPC engineer) and requires a rare combination of skills — Fortran 90, OpenACC, MPI, Cython, and modern Python packaging.

Two research directions are bottlenecked by this porting cost: (1) generating training data for neural surrogates ([Li et al., 2020](#); [Raissi et al., 2019](#)) and (2) physics-aware validation of those surrogates during training. Both require the reference solver to be fast (GPU) and callable from a Python machine-learning stack, which legacy Fortran does not natively support.

fortranspire automates the transformations a human expert performs by hand: kernel extraction from monolithic PROGRAMS into MODULEs with explicit INTENT, OpenACC pragma insertion, Cython wrapping with `iso_c_binding`, and validation through `gfortran` and `nvfortran -acc`. A run consumes four LLM calls (approximately 2 minutes wall-clock and 0.04 USD on Codestral) and produces a kernel that compiles for GPU and is importable from Python. Intermediate artefacts are written to disk between stages so a reviewer can inspect or hand-edit each step. The full pipeline architecture is documented in [the project's Sphinx site](#).

Related work

fortranspire overlaps with three lines of prior work.

Deterministic source-to-source compilers and stencil DSLs. PSyclone (UK Science and Technology Facilities Council (STFC), 2024) programmatically rewrites Fortran for OpenACC, OpenMP, and Kokkos and underpins the UK Met Office LFRic dynamical core. Loki (ECMWF, 2024), on which fortranspire builds, plays the equivalent role in the ECMWF IFS modernisation. GT4Py (Paredes et al., 2023) provides a Python-embedded stencil DSL used by Ubbiali et al. (Ubbiali et al., 2025) to port the ECMWF CLOUDSC microphysics scheme to GPU. These tools assume the input is already modular: they do not lift COMMON blocks, refactor SAVE state, or extract kernels from monolithic PROGRAMs.

Monolithic neural transpilers. CodeRosetta (TehraniJamsaz et al., 2024) is an encoder–decoder transformer trained on parallel-code pairs (Fortran C++, C++ CUDA). HPC-Coder (Nichols et al., 2024) fine-tunes a base LLM on HPC sources. Fortran2C++ (Chen et al., 2024) uses dual-agent multi-turn dialogues. Godoy et al. (Godoy et al., 2024) benchmark off-the-shelf models across HPC kernels. These approaches operate on full files in one shot, do not interleave deterministic AST analysis to bound LLM responsibility, and do not expose a tool-call interface usable from a developer’s editor.

Agentic LLM workflows. Gupta et al. (Gupta et al., 2025) propose the closest design: an agentic workflow translating legacy Fortran to Kokkos C++. fortranspire differs in (i) target — OpenACC + Cython + optional JAX rather than Kokkos, keeping the Python ML ecosystem reachable; (ii) interleaving — six of the eight stages are deterministic Loki AST work (see [architecture docs](#)), bounding LLM responsibility at three semantic edges and capping spend at four LLM calls per kernel by construction; (iii) interface — an MCP server, so any conformant client (mistral-vibe, Claude Code, Cursor) drives the pipeline without bespoke integration.

Functionality

The package exposes a unified fortranspire <verb> CLI (subcommands: analyze, explain, graph, doc, gpu, translate, profile, mcp) and a nine-tool MCP server with both stdio and HTTP/SSE transports. Four of the nine tools are deterministic and require no LLM call (analyze_kernels, explain_port_cost, build_call_graph, generate_docs). Eleven recurring Fortran patterns — INTENT, COMMON, SAVE, POINTER, AoS → SoA + collapse, stencil-vs- recurrence dependencies, ELEMENTAL, explicit KIND, LOGICAL PARAMETER flags, MPI halo exchange, and Fortran I/O — are documented in the [Fortran patterns guide](#).

Real-world demonstration

The pipeline is exercised end-to-end against PHYEX, the Météo-France physics-parameterisations package. Calling fortranspire_explain_port_cost on src/common/turb/mode_compu (119 lines, 1 routine) from inside mistral-vibe returns the structural assessment in approximately two seconds with no LLM tokens consumed (Loki AST only): 1 routine, 0 structural risks, an estimated 13 800 tokens for a full Phase-1 port, and a cost of 0.04 USD on codestral-latest. A subsequent translate_kernel_gpu call runs the OpenACC and Cython stages on the same file.

Quality control

Two test layers guard the pipeline. **Unit tests** (180+ pytest functions) cover Loki adapters, the LangGraph state-machine transitions, the MCP tool surface (a hand-rolled JSON-RPC handshake verifying the nine tools are discoverable over stdio), the structured CLI output

82 formats (SARIF 2.1.0, Markdown, JSON), and the deterministic stages independently of the
83 LLM. **End-to-end equivalence tests** compile the original Fortran and the generated OpenACC
84 variant with `gfortran -fopenacc`, execute both binaries on a deterministic input, and assert
85 `numpy.allclose` agreement within a per-kernel tolerance (`atol=1e-12`, `rtol=1e-10`). The
86 diagnostic on failure reports the maximum absolute difference and the worst-case probe
87 index. One end-to-end fixture (`wave_kernels` — two 2-D finite-difference stencils over
88 20 time steps) is currently shipped; the harness accepts additional kernels by dropping a
89 directory with `original.f90`, `openacc.f90`, `driver.f90`, and a documented tolerance into
90 `tests/fixtures/equivalence/`.

91 Limitations and roadmap

92 **Validation dependencies.** Some production codes (PHYEX, IFS) depend on ECMWF-IFS-
93 specific support modules (e.g., `YOMH00K`) not vendored with the upstream public repositories;
94 the pipeline still emits artefacts in those cases but the final `nvfortran -acc` compilation
95 gate is skipped. **JAX translation (Phase 2)** is experimental and covers a subset of kernel
96 shapes; complex time-loop patterns require manual rewriting toward `jax.lax.scan` with explicit
97 carry. **Multi-node GPU communication** is on the roadmap — GHEX (ETH Zurich CSCS and
98 [MeteoSwiss, 2023](#)) integration for GPU-to-GPU halo exchange — but not yet implemented.

99 Planned extensions include a GT4Py.next (ETH Zurich CSCS and MeteoSwiss and ECMWF
100 and DWD, 2024) code-generation target for stencil DSLs, a managed deployment on the
101 European Weather Cloud via Morpheus / OpenStack, and neural-surrogate training pipelines
102 that consume the GPU-ported kernels as the reference solver.

103 Acknowledgements

104 We thank the ECMWF Loki team for the Fortran transformation toolkit that underpins the
105 deterministic stages of this pipeline, the Université Mixte de Recherche CNRM (UMR 3589,
106 Météo-France / CNRS) for publishing the PHYEX physics package under CeCILL-C, and the
107 open-source communities behind OpenACC, JAX, Cython, LangChain / LangGraph, and the
108 Model Context Protocol.

109 References

- 110 Anthropic. (2024). *Model context protocol*. <https://modelcontextprotocol.io>.
- 111 Behnel, S., Bradshaw, R., Citro, C., Dalcin, L., Seljebotn, D. S., & Smith, K. (2011).
112 Cython: The best of both worlds. *Computing in Science & Engineering*, 13(2), 31–39.
113 <https://doi.org/10.1109/MCSE.2010.118>
- 114 Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G.,
115 Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable*
116 *transformations of Python+NumPy programs*. <https://github.com/google/jax>
- 117 Chen, L., Lei, B., Zhou, D., Lin, P.-H., Liao, C., Ding, C., & Jannesari, A. (2024). Fortran2CPP:
118 Automating fortran-to-C++ translation using LLMs via multi-turn dialogue and dual-agent
119 integration. *arXiv Preprint arXiv:2412.19770*. <https://doi.org/10.48550/arXiv.2412.19770>
- 120 ECMWF. (2023). *IFS documentation – integrated forecasting system*. <https://www.ecmwf.int/en/publications/ifs-documentation>.
- 122 ECMWF. (2024). *Loki: Source-to-source transformation framework for Fortran*. <https://github.com/ecmwf-ifs/loki>.

- ETH Zurich CSCS and MeteoSwiss. (2023). *GHEX: Generic exascale-ready library for halo-exchange operations*. <https://github.com/ghex-org/GHEX>.
- ETH Zurich CSCS and MeteoSwiss and ECMWF and DWD. (2024). *GT4Py: A python framework for high-performance domain-specific languages targeting weather and climate models*. <https://github.com/GridTools/gt4py>.
- Godoy, W. F., Valero-Lara, P., Teranishi, K., Balaprakash, P., & Vetter, J. S. (2024). Large language model evaluation for high-performance computing software development. *Concurrency and Computation: Practice and Experience*. <https://doi.org/10.1002/cpe.8269>
- Gupta, S., Kamalakkannan, K., Moraru, M., Shipman, G., & Diehl, P. (2025). From legacy fortran to portable Kokkos: An autonomous agentic AI workflow. *arXiv Preprint arXiv:2509.12443*. <https://doi.org/10.48550/arXiv.2509.12443>
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. <https://doi.org/10.1145/3600006.3613165>
- Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., & Anandkumar, A. (2020). Fourier neural operator for parametric partial differential equations. *arXiv Preprint arXiv:2010.08895*. <https://doi.org/10.48550/arXiv.2010.08895>
- Mistral AI. (2024a). *Codestral: A 22B-parameter code-specialized model with fill-in-the-middle training, 32K context, and 80+ programming language support*. <https://mistral.ai/news/codestral/>.
- Mistral AI. (2024b). *Mistral AI API platform documentation*. <https://docs.mistral.ai>.
- Mistral AI. (2024c). *Mistral Large 2: Sovereign EU flagship language model with strengthened code-generation and mathematical reasoning*. <https://mistral.ai/news/mistral-large-2407/>.
- Nichols, D., Marathe, A., Menon, H., Gamblin, T., & Bhatele, A. (2024). HPC-Coder: Modeling parallel programs using large language models. *Proceedings of the 39th International Conference on High Performance Computing (ISC HPC)*. <https://doi.org/10.23919/ISC.2024.10528929>
- OpenACC-Standard.org. (2022). *OpenACC application programming interface, version 3.3*. <https://www.openacc.org/specification>.
- Paredes, E. G., Groner, L., Ubbiali, S., Vogt, H., Madonna, A., Mariotti, K., Cruz, F., Benedicic, L., Bianco, M., VandeVondele, J., & Schulthess, T. C. (2023). GT4Py: High performance stencils for weather and climate applications using Python. *arXiv Preprint arXiv:2311.08322*. <https://doi.org/10.48550/arXiv.2311.08322>
- Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378, 686–707. <https://doi.org/10.1016/j.jcp.2018.10.045>
- TehraniJamsaz, A., Bhattacharjee, A., Chen, L., Ahmed, N. K., Yazdanbakhsh, A., & Jannesari, A. (2024). CodeRosetta: Pushing the boundaries of unsupervised code translation for parallel programming. *arXiv Preprint arXiv:2410.20527*. <https://doi.org/10.48550/arXiv.2410.20527>
- Ubbiali, S., Kühnlein, C., Schär, C., Schlemmer, L., Schulthess, T. C., Staneker, M., & Wernli, H. (2025). Exploring a high-level programming model for the NWP domain using ECMWF microphysics schemes. *Geoscientific Model Development*, 18(2), 529–546. <https://doi.org/10.5194/gmd-18-529-2025>

- 171 UK Science and Technology Facilities Council (STFC). (2024). *PSyclone: A source-to-source*
172 *fortran compiler for programmatic optimisation, parallelisation, and instrumentation of*
173 *HPC applications*. <https://github.com/stfc/PSyclone>.

DRAFT